

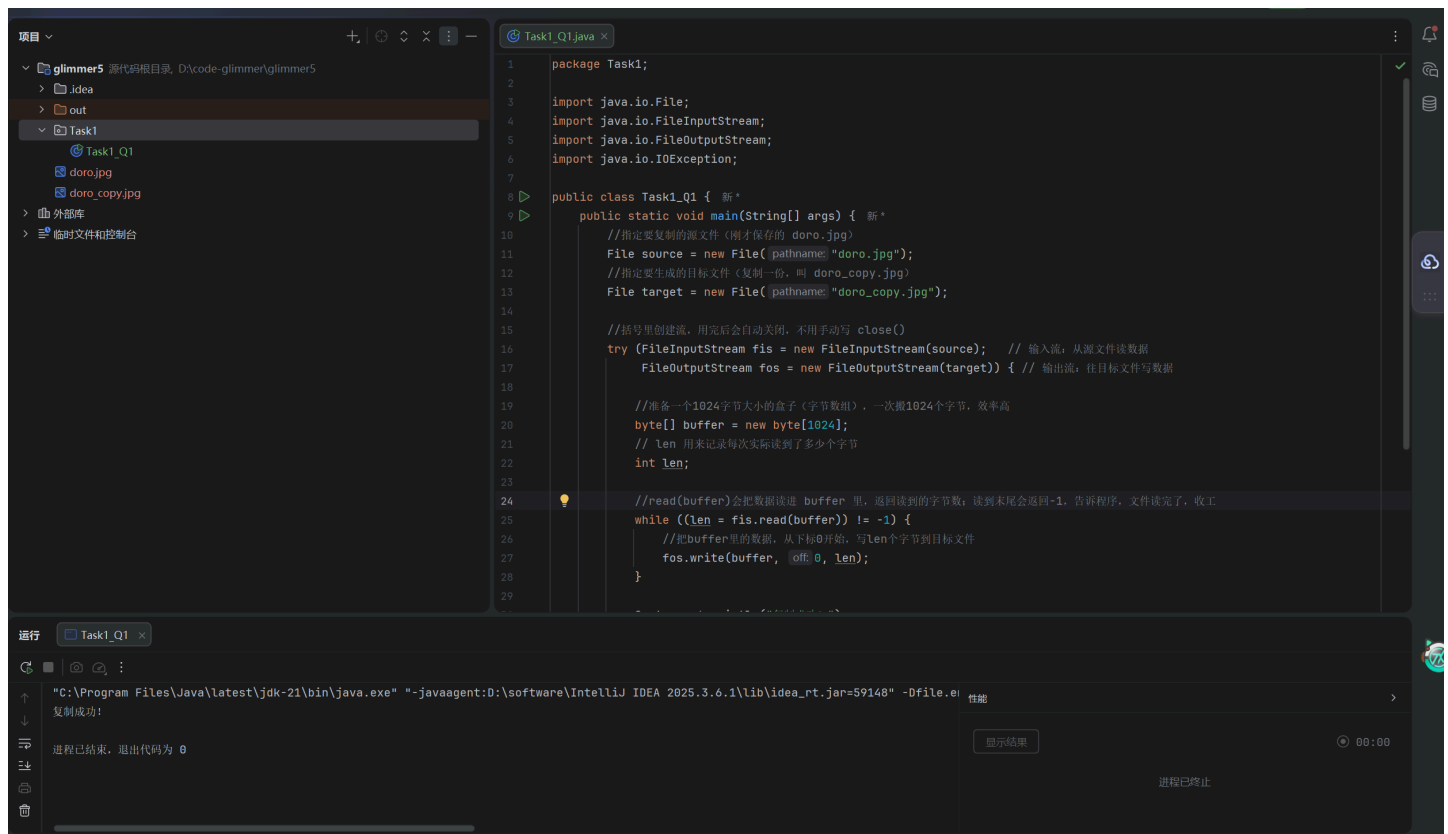
Task1

一些概念理解

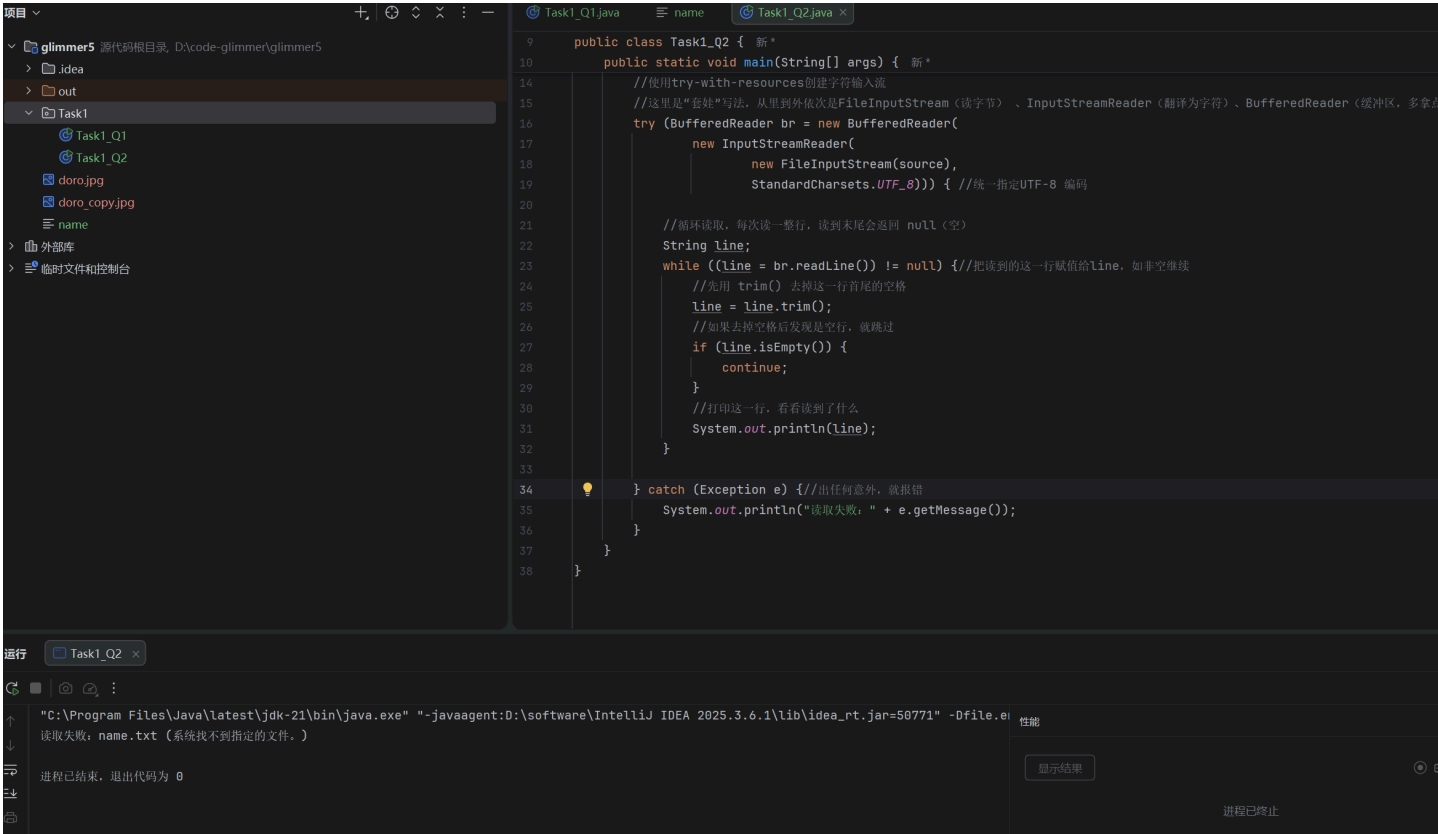
- 流：是一个动作，一个过程，像是把一桶水（源文件）通过水管转移流向另一水桶（目标文件）的“流动”过程，或者碎片一点的舀水倒水
 - 这里就和代码中的buffer有关了，一桶桶转移还是一勺一勺
- 字节流：万物皆可字节，图片、歌曲都是由0和1组成的（8个0or1为一字节），字节流就是用来搬运这些原始字节的，所以**字节流可以传输任意类型的文件**
- 字符流：字节是电脑看的，文字才是人看的，人直接看不懂字节。另外不同编码下同一个汉字对应长度数值不同（编码：文字翻译成数字的密码本，同一个字不同密码规则下字节不同，读写不一致就会乱码），如果字节流读了一半，把汉字拆开，会变成**乱码**，字符流则可以解决这个问题，**会直接在底层把字节变成字符**（根源还是在编码体系一样上），人看到的就是字符，同时可以提供一些**便捷方法**，如readLine直接读一整行，newLine自动换行等

总结：文本文件txt适合字符流，图片音频适合字节流

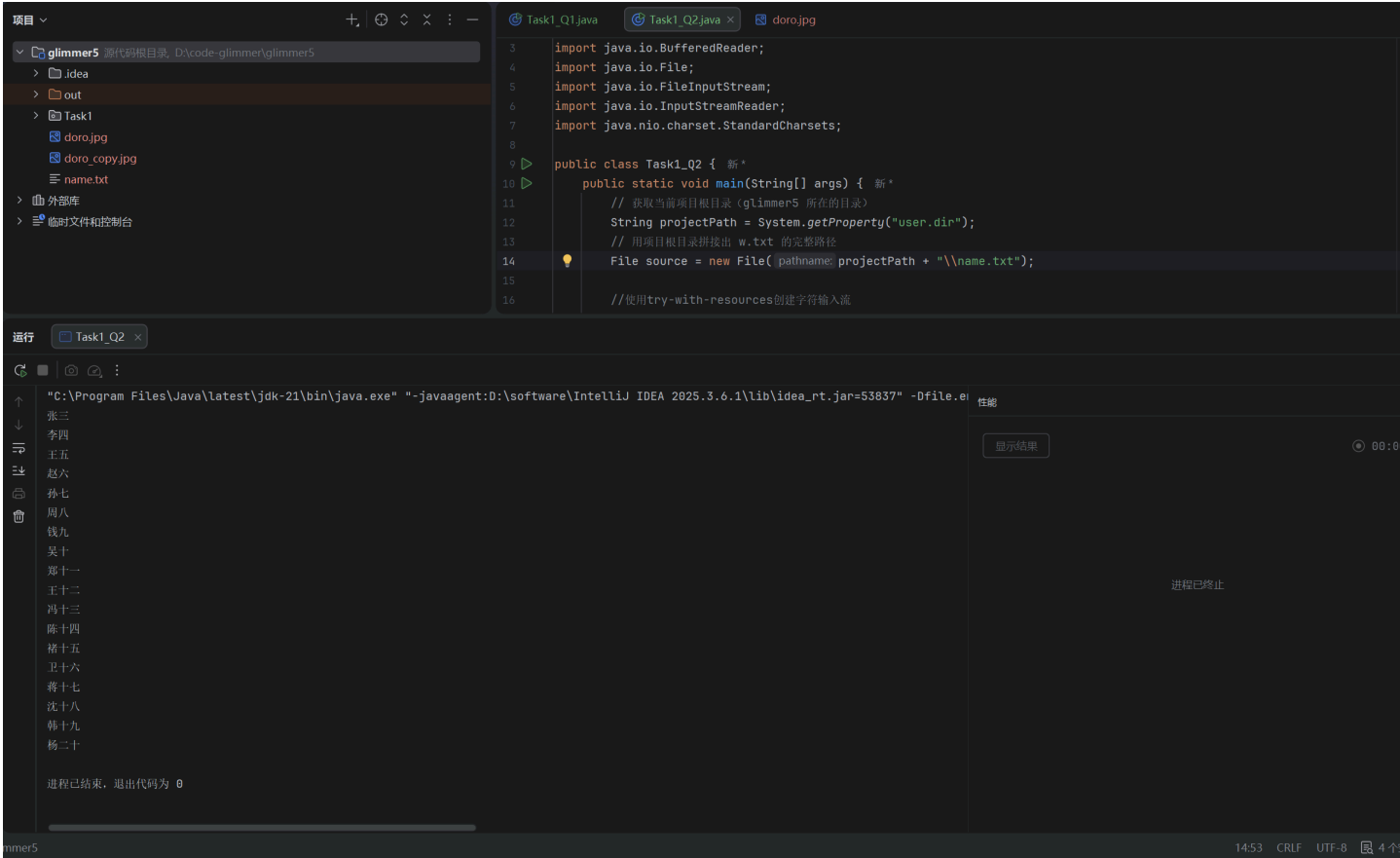
Q1



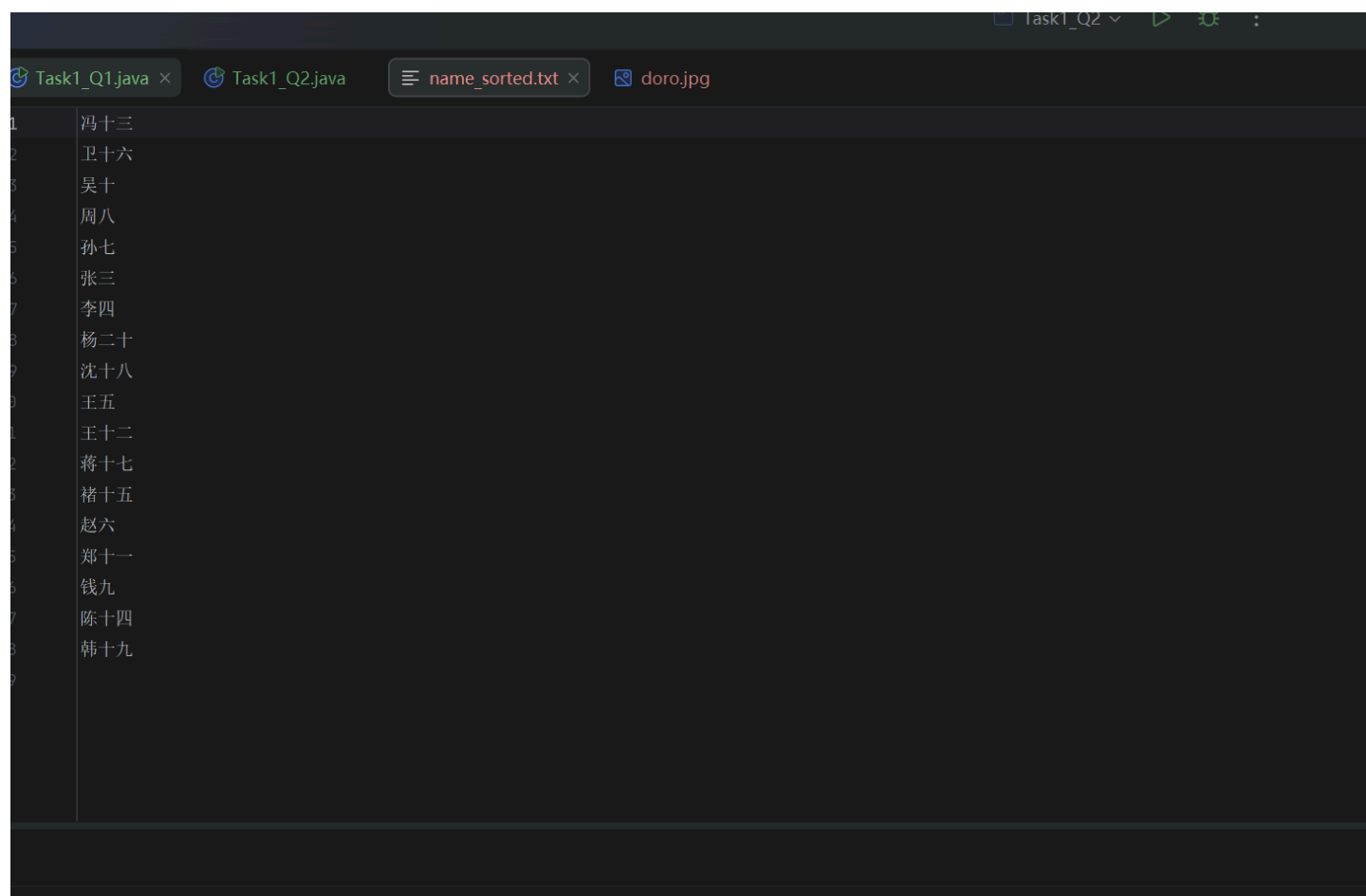
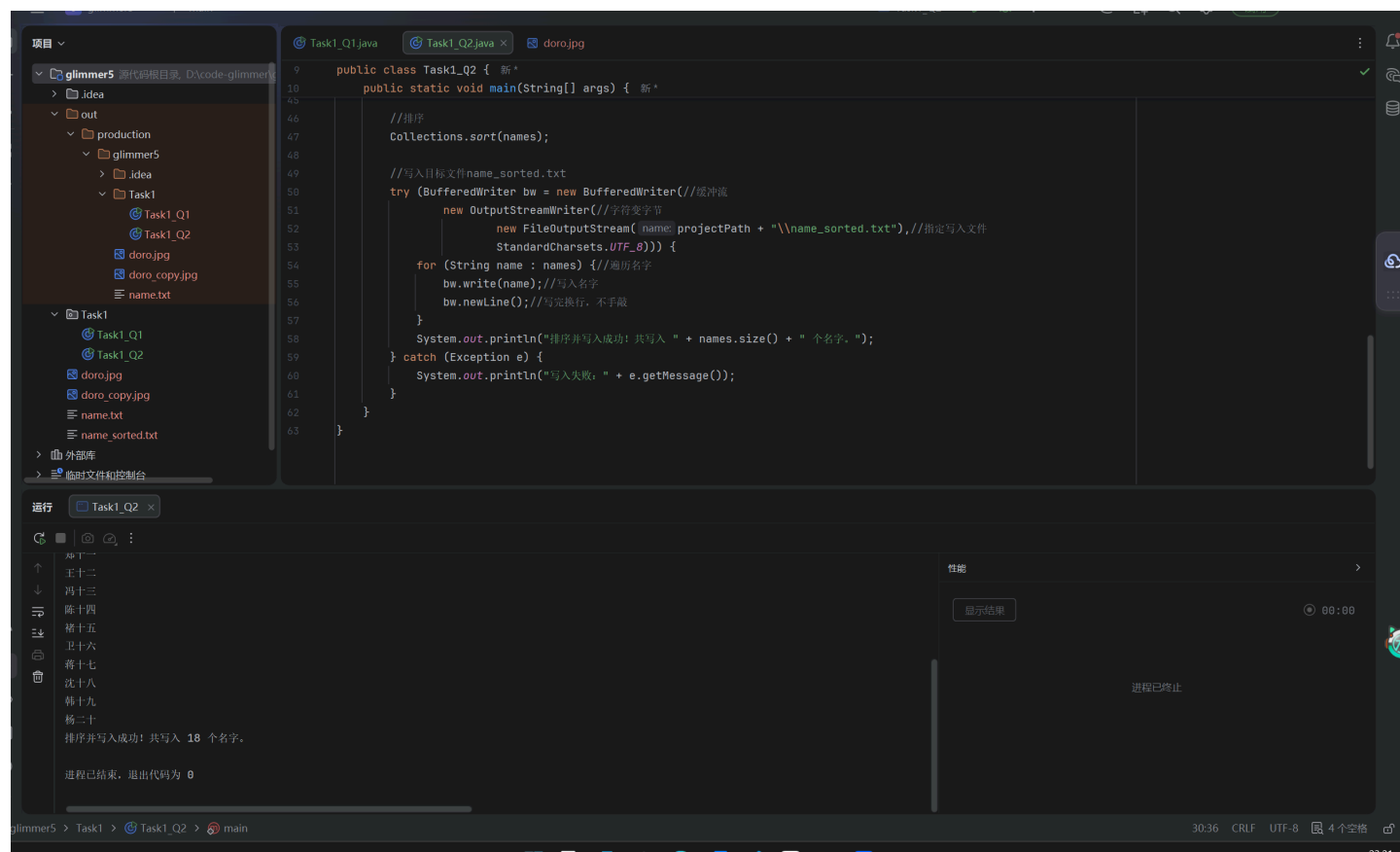
Q2



这里本来以为他会默认在根目录下找, 但是显然有问题, 题目又要求name写在根目录下, 我就改了一下代码。改好了, 没用, 干脆直接在文件资源管理器弄了一个, 暴力解决了。但是我发现这次的name后有.txt,之前都没有, 我想加上的时候还会显示文件不可用, 这应该是报错的根源, 我一开始的txt被电脑隐藏了, 也就是文件实际叫name, 但是我在代码写的是name.txt, 所以找不到文件



到这里是成功打印了，但还没在根目录生成新文件
生成过程也是有套娃，和之前思路相似，反过来，倒回去



中间多了个return，就是如果读取失败，直接退出main，不做写入，无用功

套娃式写法

BufferedReader 里面包着 InputStreamReader, InputStreamReader 里面包着 FileInputStream, 这样可以提高效率

- 最里面 FileInputStream: 负责从文件里读原始字节。
- 中间 InputStreamReader: 负责把读到的字节按照指定的编码 (UTF-8) 翻译成字符。
- 最外面 BufferedReader: 加一个缓冲流, 一次性多读点数据存着, 比一次一次单独取更快, 即加快读取速度。

null 和 isEmpty() 的区别

这两个都是“空”, 但不是一个层面的空:

- null: 是什么都没有, readLine() 读到文件末尾时返回 null, 表示没有下一行了。
- isEmpty(): 是有一个空壳, 但里面没装东西。文件里有一个空行, 读出来是一个空字符串 "", isEmpty() 就会返回 true, 表示这一行虽然是存在的, 但它是空的。

continue 注释写跳过

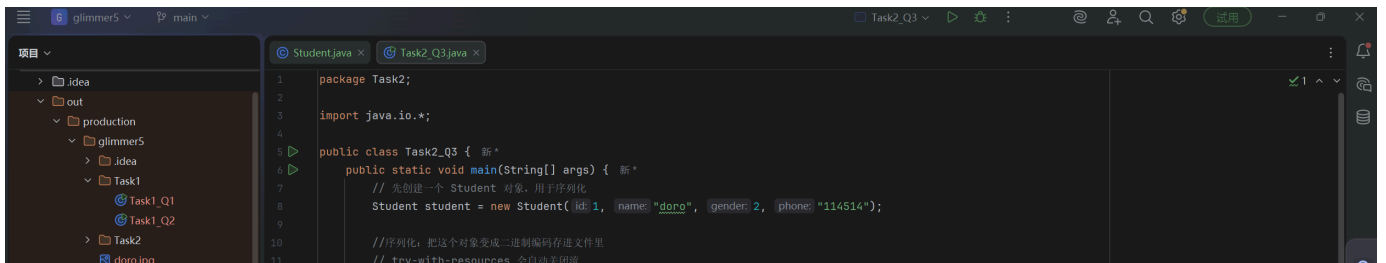
continue 英语意为继续, 意思是跳过本次循环剩下的代码, 直接进入下一次循环。当某一行是空行时, 就会跳过打印, 读取下一行

Collections.sort(names); 按编码排序, 实际效果很像按首字母, 但其实不是, 只是因为编码是参考了字典顺序

newline 换行, 自动挡, 由于系统原因, 比手敲换行更安全方便

Task2

- 序列化就是把具体对象变成一串二进制数据的过程, 可以把对象从内存到硬盘上, 永久保存。网络传输时也是序列化再反序列化的过程。
- toString: 不写则会显示地址乱码, 写后呈现成我们需要格式



- 关于流的开关：

如果写的流不关掉就开始读，可能会被上一个流限制住，读不到东西，甚至报错。
我代码中写（序列化）和读（反序列化）是独立分开的，不会冲突

Task3

- (1) maven是java项目的管理构建工具。核心突出作用在于帮助程序员自动管理项目需要的jar（第三方工具包），同时自动完成编译打包工作。是一个很省事省力的工具。
- (2) jar包，是java的压缩包，别人写好的代码，打包成.jar文件，我把这个文件引入我的项目，就可以直接调用里面的功能，不用再自己写一遍
- (3) 有关，Maven充当管家，jar包是货物资源，如果没有maven，我需要的jar包需要自己上网搜、下载、复制和一些其他复杂关系（几个程序相互联系依赖则需找全），但如果有maven，只需在配置文件里写自己需要的直接文件，maven会把这些关联文件和所需文件本身全部找全下载好
- (4) 使用maven为我们提供便利

- 可以解决前面提到的免于自己逐个下载，处理依赖冲突（A需要C的版本1.0，B需要C的版本2.0）等问题，尽量让文件都能运行，实在不行再报错给人处理
- 可以标准化项目结构（每个人拿到的项目目录都有统一格式，比如什么放主代码，什么放测试码，便于不同人交接，查找需要代码）
- 有一键构建（编译，测试，打包一条龙）的能力，不用每一步都手动操作

```
C:\Windows\system32\CMD.exe
Microsoft Windows [版本 10.0.26200.9168]
(c) Microsoft Corporation. 保留所有权利。

C:\Windows\System32>echo %PATH%
C:\Program Files\Java\latest\jdk-21\bin;D:\ASUS\apache-maven-3.9.16-bin\apache-maven-3.9.16\bin;C:\Windows\system32;C:\Windows\System32\Wbem;C:\Windows\System32\WindowsPowerShell\v1.0\;C:\Windows\System32\OpenSSH\;C:\Program Files (x86)\NVIDIA Corporation\PhysX\Common;C:\Program Files\NVIDIA Corporation\NVIDIA App\NvDLISR;C:\Program Files\dotnet\;D:\Program Files\Git\cmd;C:\Users\ASUS\AppData\Local\Microsoft\WindowsApps;D:\Users\ASUS\AppData\Local\Programs\Microsoft VS Code\bin;D:\mingw64\bin;D:\software\IntelliJ IDEA 2025.3.6.1\bin

C:\Windows\System32>mvn -v
Apache Maven 3.9.16 (2bdd9fddda4b155ebf8000e807eb73fd829a51d5)
Maven home: D:\ASUS\apache-maven-3.9.16-bin\apache-maven-3.9.16
Java version: 21.0.12.1, vendor: Oracle Corporation, runtime: D:\Program Files\Java\jdk-21.0.12.1
Default locale: zh_CN, platform encoding: UTF-8
OS name: "windows 11", version: "10.0", arch: "amd64", family: "windows"

C:\Windows\System32>
```

maven仓库

- (1) 配置maven本地仓库，主要分为建文件夹，settings标明路径，处理阿里云镜像几步，然后关联IDEA，jar包都放在这，取用时从本地拿
- (2) 中央仓库（默认的远程仓库，远程仓库还有第三方仓库如阿里云镜像，镜像中央的复制品，因为部署在国内，下载jar快，我们常用）是Maven官方维护的、全球最大的公共仓库，几乎包含所有jar包，本地没有就在这里下
- (3) 私服：公司自己内部搭建的私有仓库，存放公司自己写的代码，只给内部人员用，同时也可以代理中央仓库（需要某个包时如果私服有直接从私服获取，如没有它从中央仓库下载存在自己这里，下次要用直接获取），加速下载

创建自己的maven

- maven 项目需要配置参数：主要是 pom.xml 里的 groupId（组织名，如.alibaba或.glimmer等）、artifactId（项目名）、version（版本号），以及需要的依赖（dependencies）。
- 项目结构（规定）：
 - 代码：src/main/java
 - 配置文件：src/main/resources
 - 测试代码：src/test/java
 - 测试配置文件：src/test/resources
 - Maven 项目配置文件：根目录下的 pom.xml
- pom.xml
 - 它是Maven 项目的核心配置文件。
 - 用来管理项目信息（名字、版本）和依赖（需要哪些 jar 包）。
 - 基本配置：groupId、artifactId、version、dependencies。
 - 在依赖里写清楚 jar 包的 groupId、artifactId、version 就能管理版本；在 里可以指定项目类型（比如 jar 或 war）。

- 导入需要的 jar 包：在 pom.xml 的 标签里，写上需要的 jar 包的信息（groupId、artifactId、version），Maven 就会自动下载并引入。

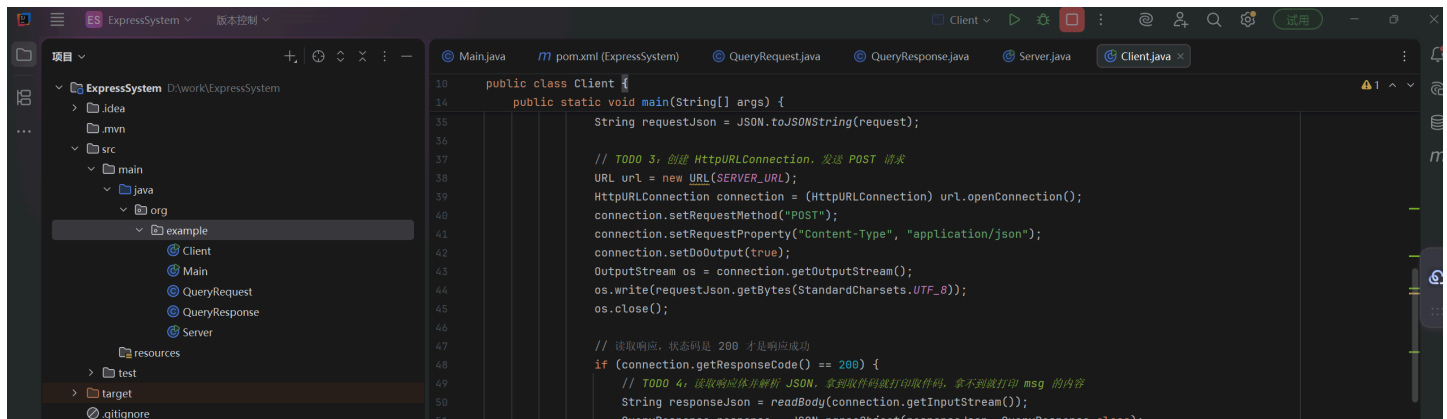
启动maven

- Maven 的常用命令：mvn compile（编译）、mvn test（测试）、mvn package（打包）、mvn clean（清理）、mvn install（安装到本地仓库）。
- IDEA 右侧的 Maven 面板里，展开 Lifecycle，可以看到 clean、compile、test、package、install 等命令。双击或者点击绿色三角，可快速执行，不用手敲

Task4

概念

- HTTP（超文本传输协议）是客户端（浏览器/APP）和服务端之间传输数据的规则。特点是：无状态（每次请求独立，不记历史）、基于请求-响应模型、支持明文（HTTP）和加密（HTTPS）传输
- JSON 是一种轻量级的数据交换格式，本质上是一个字符串。它通过“键值对”来组织数据，容易被阅读和编写，也容易被机器解析。在 Java 里，它类似于 HashMap。
 - 写完后觉得就纯是一格式，比如寄件人：xx，收件人：xx，收货地址：xx；是一种大家都认可，都能看懂的格式
- fastjson 的两个核心方法：
 - JSON.toJSONString(obj)：把 Java 对象转成 JSON 字符串。
 - JSON.parseObject(json, Class)：把 JSON 字符串转成 Java 对象。
- TCP/IP 通信：TCP/IP 是互联网的基础通信协议。HTTP 就是建立在 TCP/IP 之上的。在这个项目里，客户端和服务端通过 HTTP 协议进行通信（底层就是 TCP/IP）。



运行Client前，Server必须处于运行状态，不关闭

项目内容理解

- 使用者= Client（客户端）
- 快递柜 = Server（服务端）
- 客户的单号和手机号 = QueryRequest（请求）
- 屏幕上的取件码 = QueryResponse（响应）
- “滴”一声去数据库查 = QueryHandler（工作人员）

客户端和服务端交流

- Client：用户输入单号和手机号，把单号和手机号写在信上（QueryRequest），fastjson 把这两个数据变成一个 JSON 字符串（像填好快递单），然后通过邮局（URLConnection）寄给 Server 服务端。
- Sever
 - 收到 JSON 字符串。
 - fastjson 把 JSON 字符串变回 QueryRequest 对象。
查数据。
 - 把查询结果（取件码或错误提示）用 fastjson 变成 JSON 字符串。
 - 通过 HTTP 发回给客户端，并带上一个状态码（200 或 400）。
- Client：收到信息，打印出取件码。

写信（客户端到服务端）和回信（服务回客户），用的都是“JSON”这种格式（一种大家都看得懂的通用格式）。

为什么要用 fastjson?

- fastjson：专门帮你把 Java 对象变成 JSON 信纸，或者把 JSON 信纸变成 Java 对象。没有它，你就得自己手动拼字符串，容易出错。
-

- 状态码就是服务器告诉客户端“我处理得怎么样了”的一个数字。

在这个项目里，状态码表示：

- 200：查询成功，把取件码给你。
 - 400：你发的单号或手机号是空的，或者格式不对，我处理不了。
 - 405：你应该用 POST 来查，但你没用 POST，我不接待。
-

关于 try-catch 在客户端和服务端的兜底作用：

- 客户端：try-catch 保证了一次请求失败（比如服务端没启动、网络断了），程序不会崩溃，而是提示错误，并允许用户继续查询。因为客户端是一个循环，用户可能查询多次，不能因为一次失败就整个退出。
- 服务端：try-catch 保证了即使客户端发来的数据格式不对（比如 JSON 解析失败、手机号为空），服务端也不会崩溃，而是返回一个 400 错误提示。服务端是为多个客户端服务的，必须保证自身的稳定，不能因为一个客户端的错误就宕机。